LAB 10: Design Analysis Report

Smart Waste Collection Management System

KIET Deemed To Be University | Computer Science Engineering | CSE-E

Prepared by: Tanishka Singh - Leader (Product Manager)

1. Introduction

This report analyses the design of the Smart Waste Collection Management System. The design was produced across the previous labs as a use case diagram (Lab 2), a UML class diagram (Lab 3), activity diagrams (Lab 4), sequence diagrams (Labs 5 and 6), collaboration diagrams (Lab 7), state transition diagrams (Lab 8) and component diagrams (Lab 9). The aim here is to evaluate that design against the requirements and quality goals in the Software Requirements Specification (SRS v1.0): to check whether every requirement is covered, whether the structure follows good design principles, and where the design needs improvement.

2. System design overview

The SRS describes four major parts that talk to each other over Wi-Fi/GPRS and REST APIs: IoT sensor nodes on the bins, a cloud backend server, a web-based administrator dashboard and a mobile application for drivers and citizens. The design realises this as a layered, client-server architecture in which sensor nodes push readings to the backend and every client uses the backend only through well-defined interfaces.

Layer	Components	Responsibility
Device layer	Sensor Node Firmware (ESP32 / Arduino, HC-SR04, Wi-Fi / GSM)	Measure distance, convert it to fill percentage, retry failed reads, transmit every 15 minutes.
Service layer	Backend Server: REST API Layer, Auth, Data Ingestion, Alert, Route Optimization, Complaint and Analytics services	Business logic: store readings, check thresholds, raise alerts, compute routes, handle complaints, build reports.
Integration layer	Notification Adapter (FCM), Map Adapter (OSM / Google Maps)	Hide third-party services behind internal interfaces.
Data layer	Data Access Layer, Database (MongoDB / PostgreSQL)	Store bins, readings, alerts, routes, complaints and users.
Client layer	Admin Web Dashboard, Driver Mobile App, Citizen Mobile App / Web Form	User interaction over HTTPS / JSON.

3. Summary of the design models

Lab	Model	What it covers
2	Use case diagram	14 use cases and 7 actors (4 human, 3 system) with preconditions, postconditions and functions.
3	Class diagram	11 classes: User (abstract), Administrator, Driver, Citizen, Bin, SensorNode, SensorReading, Alert, Route, Complaint, Report, with 13 links (3 generalisations, 10 associations, aggregation and dependency).
4	Activity diagrams (2)	End-to-end monitoring, alert, routing and collection flow with swimlanes; citizen complaint flow with fork and join.
5, 6	Sequence diagrams (5)	Monitoring and alert, route and collection, complaint handling, admin login with analytics, threshold configuration.
7	Collaboration diagrams (3)	The same monitoring, routing and complaint scenarios as numbered object interactions.
8	State transition diagrams (4)	Life cycle of Bin, Sensor Node, Route and Complaint.
9	Component diagrams (2)	System-level components and interfaces; internal modules of the backend.

4. Design analysis

4.1 Class design: cohesion and coupling

Each class has one clear purpose, which gives high cohesion: SensorReading only holds one measurement, Alert only records and sends a notification, Report only represents analytics. The three user roles inherit login, logout and credentials from the abstract User class and add only role-specific operations, so the hierarchy follows the is-a rule and avoids duplicated code. The table below counts the direct associations of each class (generalisation not counted).

Class	Direct links	Linked to	Observation
Bin	4	SensorNode, Alert, Route, Complaint	Hub of the model. Correct, because every feature revolves around the bin, but it must not collect extra logic.
Route	3	Bin, Administrator, Driver	Aggregates bins; route logic stays inside Route.
SensorNode	2	Bin, SensorReading	Isolated hardware side of the system.
Alert	2	Bin, User	Loosely coupled to users through a dependency only.
Complaint	2	Citizen, Bin	Resolved through Bin, which implements rule 5.5.
Administrator	2	Route, Report	Report, Driver, Citizen, SensorReading and User have one link each.

Overall coupling is low: the longest dependency chain is Citizen, Complaint, Bin, SensorNode, SensorReading. Bin is the only class with four links and should be watched during implementation so that it does not become a god class (see section 7).

4.2 Design principles

- **Single Responsibility:** the backend is split into separate services (ingestion, alerts, routing, complaints, analytics, auth), matching the modularity requirement in SRS 5.4.
- **Open / Closed:** new bin types, zones or report types can be added by adding classes or services without changing existing ones (SRS 5.4 scalability).
- **Liskov Substitution:** Administrator, Driver and Citizen can be used wherever a User is expected, for example when an Alert notifies users.
- **Interface Segregation:** clients use small interfaces (IDashboardAPI, IRouteAPI, IComplaintAPI) instead of one large API, so the driver app never depends on dashboard operations.
- **Dependency Inversion:** services depend on internal interfaces (INotify, IMap, IStorage), not directly on FCM, OpenStreetMap or the database product.
- **Separation of concerns:** hardware, business logic, integration, data and presentation are in different layers.

4.3 Design patterns identified

Pattern	Where it appears	Benefit	SRS link
Layered architecture	Device, service, integration, data and client layers	Changes stay inside one layer	2.1, 5.4
Facade	REST API Layer in front of all services	One entry point for dashboard and mobile app	3.3, 3.4
Adapter	Notification Adapter (FCM), Map Adapter (OSM / Google Maps)	Provider can be swapped without touching services	TBD-2, 2.5
Repository	Data Access Layer	Services are independent of MongoDB / PostgreSQL	3.3
Observer / publish-subscribe	Alert triggers push notifications to admin and driver	Alert source does not know the receivers	REQ-AL-2
Strategy (recommended)	Route computation: nearest-neighbour vs VRP solver	Algorithm can be chosen later	TBD-1, REQ-RO-2

4.4 Behavioural analysis

The activity, sequence and state diagrams agree with each other and with the SRS. Four decision points control the main behaviour: reading valid, fill-level at or above threshold, adjust route, and more bins on route. The complaint flow uses a fork and join so that the citizen and the administrator work in parallel. Each state machine has an initial state, every state has an outgoing transition, and the Bin machine returns to Normal after collection, so no object can get stuck. Business rule 5.5 (a collected bin resolves its complaint) appears in the Bin class, the route sequence diagram and the Complaint state machine, which shows that it is applied consistently.

4.5 Component and interface analysis

The component diagrams show only interface-based connections, with no component calling the internals of another. Seven interfaces at system level and about ten inside the backend keep coupling low and let each component be tested separately by the tester role. The main structural risk is that all clients and sensors depend on one backend component, which is covered in section 6.

5. Requirements traceability

Requirement	Class diagram	Behaviour diagrams	Component
REQ-BM-1 to 3 (monitoring)	SensorNode, SensorReading, Bin	Sequence 1, Collaboration 1, State 2, Activity 1	Data Ingestion Service
REQ-AL-1 to 3 (alerts)	Alert, Bin, Administrator	Sequence 1, Sequence (Lab 6) 2, Collaboration 1, State 1	Alert Service, Notification Adapter
REQ-RO-1 to 3 (routes)	Route, Bin, Administrator, Driver	Sequence 2, Collaboration 2, State 3, Activity 1	Route Optimization, Map Adapter
REQ-DB-1 to 3 (dashboard)	Report, Administrator	Sequence (Lab 6) 1	Analytics Service, Auth Service, Admin Dashboard
REQ-CF-1 to 3 (complaints)	Complaint, Citizen	Sequence 3, Collaboration 3, State 4, Activity 2	Complaint Service, Citizen App
SRS 5.3 (security)	User (credentials)	Sequence (Lab 6) 1 and 2: token, role check, 401 / 403	Auth Service, HTTPS interfaces
SRS 5.5 (business rules)	Bin.isAboveThreshold(), Bin.markCollected()	Sequence 2, State 1 and 4	Route Optimization, Complaint Service

Every functional requirement from the SRS is covered by at least one class, one behaviour diagram and one component.

6. Quality attribute analysis

Attribute	SRS requirement	How the design supports it	Residual risk
Performance	500 concurrent sensors; dashboard in 3 s; route in 10 s for 200 bins	Separate ingestion and route services; Strategy for the algorithm	Must be verified by load testing in the test phase
Reliability	95% successful transmission	Retry and log in Sensor Node (Error Handling state); acknowledgement from server	Local buffering during GPRS outages is not modelled
Scalability	Add bins and zones without redesign	Stateless services and repositories	Single backend is one point of failure
Maintainability	Modular code	Services, adapters and layers	Keep Bin free of extra logic
Security	HTTPS / TLS; admin authentication; protect citizen data	Auth Service with role check, OTP option, 401 / 403 responses	Retention rules for citizen data are not defined
Usability	Simple app for drivers and citizens	Small interfaces per client; few steps in each flow	Needs user testing
Safety	System must not block manual collection	Collection works from route and manual edits; no hard dependency in the flows	Drivers need a fallback when the app is offline

7. Design issues and recommendations

No.	Issue found	Recommendation	Priority
1	Bin has four links and may attract too much logic.	Keep threshold checks in Alert Service and route selection in Route Optimization; Bin holds data and simple state changes.	Medium
2	Alerts have no acknowledgement or escalation.	Add an acknowledged status to Alert and a reminder if nobody reacts.	Medium
3	The SRS notes weak GPRS coverage, but the sensor design has no local buffer.	Store readings in the node and send them in a batch when the link returns.	High
4	Bin state machine has no state for a faulty sensor or a bin taken out of service.	Add Out of Service and Sensor Fault states with transitions to and from Normal.	Medium
5	Route algorithm is still open (TBD-1).	Define a RoutingStrategy interface now and test nearest-neighbour first.	High
6	One backend instance serves all clients.	Keep services stateless so that several instances can run behind a load balancer.	Medium
7	Retention and deletion of citizen data are not defined.	Add a retention rule and restrict photo and location access to administrators.	Medium

8. Open items from the SRS and how the design handles them

SRS item	Design provision
TBD-1: route optimization algorithm	Hidden behind the Route Optimization service; a Strategy interface allows either option.
TBD-2: mapping API provider	Hidden behind the Map Adapter (IMap).
TBD-3: default threshold	Stored as a configurable value per bin or global (REQ-AL-1), so the pilot result can be applied without code changes.

9. Conclusion

The design is consistent across all models and covers every functional requirement of SRS v1.0. It has low coupling, high cohesion, a clear layered structure and interface-based components, and it leaves the three open SRS decisions flexible. The most important improvements are local buffering on the sensor node, an extra state for faulty or out-of-service bins, alert acknowledgement and a defined data-retention rule. These can be added without changing the overall structure, so the design is ready for the implementation and testing phase. The design was also implemented as a working prototype (see Lab 11) with the same classes, services and module structure, and its 37 automated tests pass, which confirms that the design can be built as described.